LLMProvider

LLMProvider

One interface, 43 providers — with circuit breakers, retries, and health baked in.

Summary

LLMProvider is a generic, reusable Go module that defines a unified `LLMProvider` interface plus the production resilience patterns around it — circuit breaker, health monitoring, retry with backoff, lazy loading — and ships 43 concrete provider implementations behind that one contract, with an OpenAI-compatible generic adapter and honest, no-hardcoded-fallback model discovery.

Short description

A reusable Go module exposing one `LLMProvider` interface (`Complete`, `CompleteStream`, `HealthCheck`, `GetCapabilities`, `ValidateConfig`) plus fault-tolerance primitives — circuit breaker, health monitor, jittered-backoff retry, lazy init — over 43 provider adapters and a generic OpenAI-compatible adapter. Thread-safe.

Long description

LLMProvider is the abstraction layer every LLM-consuming service needs but almost nobody builds well — the unglamorous plumbing that separates a demo from a system that survives contact with real traffic. It defines a single, capability-aware interface — `Complete`, `CompleteStream`, `HealthCheck`, `GetCapabilities`, `ValidateConfig` — so application code targets exactly one contract no matter which of 43 backends answers the call, and then it ships the operational hardening that turns fragile provider calls into something you can run in production without holding your breath. A three-state circuit breaker (closed → open → half-open) transparently wraps any provider — *including its streaming*

channel, where an empty stream is correctly counted as a failure — so a single misbehaving backend can trip open and stop taking the whole service down with it; a central `CircuitBreakerManager` tracks every breaker at once. A configurable health monitor continuously walks providers through healthy / degraded / unhealthy / unknown states on threshold-and-interval checks, so degradation is observed rather than discovered by an outage. Retry logic layers exponential backoff with jitter on top, making status-aware decisions — retry the errors worth retrying (429, 5xx, transient network faults), never burn cycles on 4xx or a cancelled context — with delays clamped so a backoff storm can't run away. And a lazy-init pattern defers each provider's construction until its first real use — a deliberate design choice that keeps registration of all 43 providers essentially free.

The module ships 43 concrete provider packages plus a `generic` OpenAI-compatible adapter that implements the full interface against *any* `/v1/chat/completions` endpoint — Bearer auth, SSE streaming with correct `[DONE]` handling — so a vendor without a dedicated package is still a first-class citizen the moment you point the adapter at its URL. Credentials are resolved in exactly one place (`apikeys`, using a strict `ApiKey_<Provider>` convention), closing off the whole "hardcoded key passes the test suite, the real key was never wired, product breaks in prod" class of bug at the source. Model discovery is deliberately, almost stubbornly honest: it queries live provider APIs behind a TTL cache, and — per governance — the old hardcoded fallback tier was deleted outright. When live discovery fails, LLMProvider returns *nothing* rather than a stale catalogue, so a caller is never handed a model ID that looks valid and then can't be invoked. Every piece of this is built thread-safe for concurrent use.

Why we built it

Naive LLM calls fail in production — providers rate-limit, degrade, or go down, and one bad backend can take a service with it. Model catalogues drift, and hardcoded lists hand callers IDs that no longer work. LLMProvider centralizes the interface, the resilience patterns, and honest discovery so every consumer inherits fault tolerance and truthfulness for free.

Why it's a game-changer

It collapses "integrate an LLM provider" down to a single move — implement one interface, or just point the generic adapter at an endpoint — and then wraps that provider, automatically and transparently, in circuit breaking, health monitoring, and jittered-backoff retry. Resilience stops being something each team reinvents (badly, under deadline, after the first outage) and

becomes the library's default behaviour across all 43 backends. The reliability engineering is written once, tested hard, and inherited for free by everyone who imports it.

What's innovative

- **One capability-aware interface** — completion, streaming, health, capabilities, and config validation collapsed into a single contract every backend honours identically.
- **Transparent circuit-breaker wrapping — including streams.** The breaker protects `CompleteStream`'s channel, not just request/response, and treats an empty stream as the failure it really is — with deadlock-safe, off-lock listener notification.
- **43 provider packages + a generic OpenAI-compatible adapter** — dedicated packages stay thin, and any unlisted vendor speaking `/v1/chat/completions` works the instant you aim the adapter at it.
- **Single credential authority (`apikeys`)** — exactly one place reads `ApiKey_<Provider>` env vars, structurally eliminating the "green tests, broken product" mismatch instead of merely warning about it.
- **Honest model discovery (no hardcoded fallback)** — live provider APIs behind a TTL cache; on failure it returns `nil`, never a stale or fabricated catalogue that hands out un-invokable IDs.
- **Lazy init with `sync.Once`** — construction is deferred to first use, so registering all 43 providers costs almost nothing until you actually call one.
- **Anti-bluff, multi-locale Challenge stack** — a real runner that exercises circuit, health, and retry behaviour across five locales, gated by paired mutation testing (unmutated code must exit 0; an injected mutation must force exit 99), so a passing suite provably means working behaviour.

Biggest technical challenges & how we solved them

- **Cascading provider failures.** One flaky backend must not drag a whole service down with it. Solved with a three-state circuit breaker (closed → open → half-open) that transparently wraps any provider *and its stream*, trips open on sustained failure, probes for recovery in half-open, and is coordinated centrally by a `CircuitBreakerManager`.
- **Transient errors and rate limits.** Solved with status-aware exponential backoff plus jitter — `min(InitialDelay·Multiplier^(n-1), MaxDelay) ± jitter` — so retries spread out instead of synchronising into a thundering

herd. It retries exactly what should be retried (429, 500, 502, 503, 504, and network errors) and refuses to waste attempts on a cancelled context or any other 4xx.
- **Scaling to many registered-but-unused providers.** With 43 providers registered but only a handful live in any given service, eager construction would be pure waste. Solved with lazy initialization guarded by `sync.Once`, so only the providers you actually call ever pay their setup cost.
- **Handing out invalid model IDs.** Solved by deleting the hardcoded discovery fallback tier outright (per CONST-036) and returning nothing on live-discovery failure — plus a defensive copy-on-return so a caller can't mutate the cache or race another reader. Truthfulness is enforced structurally, not by convention.
- **Streaming + concurrency correctness.** The subtle failure mode is a deadlock between the breaker's lock and its listener callbacks. Solved by snapshotting listeners and notifying them off-lock under a 5-second timeout, and by unlocking before notifying on reset — with every component built for concurrent use and pinned by the `-race` suite.

Tech stack

- **Go (1.25.3)** — chosen for first-class concurrency, static binaries, and a strong standard library; carries the module, the interface, every resilience primitive, and all 43 adapters.
- **`net/http` (stdlib)** — deliberately dependency-free HTTP: powers the per-provider clients, the generic OpenAI-compatible adapter, and the live discovery calls, so there's no third-party transport to audit or patch.
- **logrus** — structured, level-aware logging exactly where operators need visibility: inside the circuit breaker's state transitions and the discovery path.
- **testify** — drives the test suite and, crucially, the mutation-branch pinning that makes a green run mean something.
- **yaml.v3** — parses the i18n bundles and configuration in a format that stays human-editable.
- **`digital.vasic.models`** — the shared `LLMRequest` / `LLMResponse` / `ProviderCapabilities` types, kept in one place so every adapter speaks the same vocabulary (a documented runtime dependency).
- **First-party packages** — `circuit`, `health`, `retry`, `apikeys`, `discovery`, `providers/` (43 vendors + `generic`), and `i18n`: the resilience and integration surface split into small, independently testable units rather than one monolith.
- **`.env` + `~/api_keys.sh` (`ApiKey_<Provider>` convention)** — a single, unambiguous source of credential truth, so keys are wired the same way in tests and in production.
- **Makefile race suite (`-race -p 1`) + Challenge runner** — the anti-bluff backbone: the race detector proves concurrency

correctness, and the Challenge runner hammers real behaviour through chaos, ddos, scaling, stress, live-discovery, and no-suspend scenarios.

Status & honesty notes

- **Status: beta.** A decoupled reusable module; the GitHub repo is public.
- **License: TBD.** Inconsistent — `doc.go` says MIT while an Apache-2.0-style LICENSE file is present — verify before publishing.
- LLMsVerifier is the upstream single source of truth for the canonical model catalogue. The `helix-deps.yaml` manifest appears stale (declares `deps: []` while docs state a dependency on `digital.vasic.models`); discovery's "Tier 2 (models.dev)" is a planned stub, not active.

Priority tier: Helix-primary (LLM-infrastructure cluster — decoupled reusable module). Ranks after HelixTrack.